

Fundamentals of Functional Programming Languages

Introduction

Functional Programming is a programming paradigm where computation is performed by **evaluating mathematical functions** rather than executing step-by-step instructions.

Functional programming languages are mainly designed to handle:

- Symbolic computation
- List processing applications

The main goal of Functional Programming Languages (FPL) is to **mimic mathematical functions as closely as possible**.

Some popular functional programming languages include:

- Lisp
- Haskell
- Python
- Erlang
- Clojure

Types of Functional Programming Languages

Functional languages are classified into two types.

1. Pure Functional Languages

These languages follow **only functional programming concepts**.

Example

- Haskell
-

2. Impure Functional Languages

These languages support both:

- Functional programming
- Imperative programming

Example

- Lisp

These languages allow variables and other programming constructs.

Characteristics of Functional Programming

1. Based on Mathematical Functions

- Programs are written using mathematical expressions.
 - Example: $f(x) = x * x * x$
 - This represents a cube function.
-

2. Higher-Order Functions

- A higher-order function is defined as a function that can:
- Accept other functions as input parameters
- Provide a function as its resulting output

- Common operational examples include:
 - Map
 - Filter
 - Reduce
-

3. Lazy Evaluation

Within this paradigm, expressions are **evaluated only upon necessity**, which serves to enhance computational efficiency.

4. Absence of Conventional Control Flow

Functional programming languages typically use:

- for loops
- while loops
- switch statements

They rely instead upon:

- Recursion
 - Function application
-

5. Referential Transparency

In the functional context, a specific function will **consistently produce an identical result** when provided with the same input values.

Example: $f(5) = 25$ // The output remains invariable across executions.

Mathematical Functions

A mathematical function maps elements from one set to another.

- Input set → **Domain**
- Output set → **Range**

Example

$$\text{cube}(x) = x * x * x$$

Lambda Expression

Lambda expressions are used to define functions.

Example

$$(f(x) x * x * x)(3)$$

Output

27

Functional Forms

Functional forms are **higher-order functions** that operate on other functions.

1. Function Composition - Combines two functions.

$$h(x) = f(g(x))$$

Here:

- $g(x)$ executes first
 - $f()$ is applied to the result.
-

2. Construction - Applies multiple functions to a value.

Example

$$f(x) = x * x * x$$

$$g(x) = x + 3$$

$$\rightarrow (64, 7)$$

3. Apply-to-All - Applies a function to every element of a list.

Example

$$h(x) = x * x * x$$

$$f(h, (3,2,4)) \rightarrow (27,8,64)$$

Advantages of Functional Programming

- Simpler program structure
- Easier debugging
- Supports parallel processing
- More reliable programs

LISP Language

Lisp is the **first functional programming language**.

It has two main forms:

1. Data Objects

Originally LISP had only two types:

- Atom
 - List
-

2. List Form

Lists are written using parentheses.

Example

(A B (C D) E)

Lists are internally stored as **single linked lists**.

If interpreted as:

Data → list of atoms

Function → function call

Example

(A B C)

means function **A** applied to **B** and **C**.

Functional vs Imperative Languages

Feature	Imperative Languages	Functional Languages
Approach	Step-by-step instructions	Based on mathematical functions
Variables	Uses variables and updates values	No variable modification (immutable)
Control	Uses loops (for, while)	Uses recursion and functions
Side Effects	Has side effects	Avoids side effects
Examples	C, Java	Haskell, Lisp

Programming with ML (Meta Language)

- ML stands for **Meta Language**. It is a **functional programming language** introduced by **Robin Milner in 1997**.
 - ML is mainly used for symbolic computation and functional programming. Like Scheme language, ML uses **static scoping**.
-

Characteristics of ML

1. Strongly Typed Language

- Every variable and expression has a fixed type.
- Wrong type operations are not allowed.

Example

5 + "Hello"

This produces an error because integer and string cannot be added.

2. Type Inference

ML can automatically identify the type of variables and expressions.

Example

val x = 10;

ML automatically treats **x** as integer type.

3. Identifiers Instead of Variables

ML does not use variables like C or Java. Instead, it uses **identifiers** which act as names for values.

Example

```
val a = 5; // Here a is an identifier.
```

Syntax and Functions in ML

1. Infix Notation - ML uses infix notation for arithmetic operations.

Example: $7 + 3$

2. Function Declaration - Functions are declared using the **fun** keyword.

Example: `fun add(x,y) = x + y;`

3. Default Types - If type is not specified, ML automatically assigns suitable type. Usually integer values are inferred as integer type.

4. Manual Type Declaration - Types can also be declared manually using colon `::`.

Example:

```
fun area(x:real) = 3.14 * x * x; //Here x is declared as a real type.
```

Example

```
fun square(x) = x * x;
```

Supported Structures in ML

ML supports:

- Enumerated types
 - Arrays
 - Tuples
 - Exception handling
 - Module facilities for Abstract Data Types (ADTs)
-

Important Rules in ML

1. No Coercion

ML does not automatically convert one data type into another.

Example

```
fun add(x:int) = x + 1;
```

Calling:

```
add(5.5)
```

produces type mismatch error.

2. Return Value Rule

If multiple expressions are separated by semicolons, ML returns the value of the last expression.

Example

```
(3+2;  
4+5;  
10+1;)
```

Output

```
11
```

Advantages of ML

- Simple and easy to understand
- Strong type checking reduces errors
- Supports functional programming
- Automatic type inference
- Supports reusable functions

Multi-Paradigm Languages

A multi-paradigm language is a programming language that supports more than one programming style or paradigm.

It allows programmers to use different methods such as:

- Procedural programming
- Object oriented programming
- Functional programming
- Logic programming

in the same language.

Features of Multi-Paradigm Languages

1. Supports Multiple Programming Styles - One language can support many paradigms. **Example:** Python

2. Code Reusability - Programs can be reused easily using functions, classes, and modules.

3. Flexibility - Programmers can choose the best programming method for solving problems.

4. Easy Maintenance - Programs become easier to modify and debug.

Examples of Multi-Paradigm Languages

1. Python

Supports:

- Procedural
- Object oriented
- Functional programming

Example

```
x = 10
```

```
print(x)
```

2. Java

Supports:

- Object oriented
- Generic programming

Example

```
class Student
```

```
{
```

```
}
```

3. C++

Supports:

- Procedural
- Object oriented
- Generic programming

Example

```
class A
{
};
```

4. JavaScript

Supports:

- Event driven
 - Functional
 - Object oriented programming
-

Common Paradigms Supported

Paradigm	Description
Procedural	Program divided into functions
Object Oriented	Program based on classes and objects
Functional	Program based on functions
Logic	Program based on facts and rules

Advantages of Multi-Paradigm Languages

1. Flexible programming
 2. Better code reuse
 3. Easy software development
 4. Reduces program complexity
 5. Suitable for large applications
-